

Then the adjacency list of this diagram in Lisp may be expressed as:

```
;; Graph as adjacency list
(defparameter *example-graph*
  '((A B C)
    (B A D E)
    (C A F)
    (D B)
    (E B F)
    (F C E)))
```

It is now necessary to have a function with 'assoc' to determine a node's nearby neighbours, as indicated by the code below:

```
(defun neighbors (node graph)
  (cdr (assoc node graph)))
```

Now we can define the BFS Lisp script for the aforementioned algorithms:

```
(in-package: pathfinding)
;;; BFS
(defun bfs (start goal graph)
  (let ((queue (list (list start)))
        (visited '()))
    (loop while queue do
      (let* ((path (pop queue))
             (node (car (last path))))
        (unless (member node visited)
          (if (equal node goal)
              (return path)
              (progn
               (push node visited)
               (dolist (n (neighbors node graph))
                 (push (append path (list n)) queue))))))))))
```

The Lisp function for the DFS algorithms can also be written as:

```
(in-package :pathfinding)
;;; DFS
(defun dfs (start goal graph &optional (path '()) (visited '()))
  (setq path (append path (list start)))
  (cond
   ((equal start goal) path)
   ((member start visited) nil)
   (t
    (dolist (n (neighbors start graph))
      (let ((result (dfs n goal graph path (cons start
```

```
visited))))
      (when result (return result))))))
```

For the implementation of A* algorithms, we need one more piece of information about a connected graph: the coordinates of each node or the cost of each edge. In this discussion, we shall consider the coordinates only to measure the Euclidean distances between the nodes to select the shortest path from the origin to the goal.

In two dimensions, if the coordinates of the six nodes [A, B, C, D, E, F] are [(0,0), (1,0), (0,1), (2,0), (1,1), (0,2)], respectively, then the positions of the nodes can be expressed as:

```
(defparameter *example-positions*
  '((A (0 0)) (B (1 0)) (C (0 1))
    (D (2 0)) (E (1 1)) (F (0 2))))
```

To measure the Euclidean distance between the nodes, a function 'euclidean' can be written as:

```
(in-package: pathfinding)

(defun euclidean (a b positions)
  "Compute Euclidean distance between nodes A and B using
POSITIONS alist."
  (let ((pa (cadr (assoc a positions)))
        (pb (cadr (assoc b positions))))
    (sqrt (+ (expt (- (first pa) (first pb)) 2)
              (expt (- (second pa) (second pb)) 2)))))
```

With these two pieces of code, we can go for the implementation of the heuristic algorithm A*. The code is a bit complex, but a close observation can reveal its intricacy. Here's the A* Lisp script:

```
(defun a-star (start goal graph &optional (positions
*positions*))
  "A* pathfinding from START to GOAL in GRAPH using Euclidean
distance."
  (let ((open (list (list start)))
        (visited '()))
    (loop while open do
      (setf open (sort open #'<
                       :key (lambda (path)
                              (+ (length path)
                                 (euclidean (car (last path))
                                             goal positions))))))
      (let* ((path (pop open))
             (node (car (last path))))
        (unless (member node visited)
          (if (equal node goal)
              (return path)
              (progn
               (push node visited)
               (dolist (n (neighbors node graph))
                 (let ((result (a-star n goal graph positions)))
                   (when result (return result))))))))))
```